

Production Information System

CHAPTER 2: DataBase Modeling and Design - The Relational DataBase Model

Dr. Jomana Bashatah

Outline

- Introduction
- The DataBase Management System (DBMS)
- The Relational DataBase (RDB)
- Key Attributes and Linking Tables
- Structured Query Language (SQL)

5. Structured Query Language

- CREATING THE DATABASE AND TABLE STRUCTURE
- The CREATE command keyword is used for creating databases and tables. The ANSI syntax for database creation is
- CREATE DATABASE <database name>;
- The syntax for creating a table within a database uses the CREATE keyword as follows:
- CREATE TABLE <table name>
(<[attribute1 name]> <data type>,
<[attribute2 name]> <data type>,
... ..
<[attributeN name]> <data type>);
- When the table is created, the columns will be ordered by the attribute name order given in the CREATE TABLE command

5. Structured Query Language

- Access Exercise 2.1:
- We wish to establish a database for the tables of Figure 2.1. We will use the database title MATERIAL_MANAGER. Establish the database by following these steps:
- From the start menu, launch Microsoft Office Access 2003.
- From the File Menu, select New. From the New File selection panel, select Blank Database. The File New Database dialog window will open.
- Type in the database name: MATERIAL_MANAGER. Click the CREATE command button and a database by that name is established.

5. Structured Query Language

- Access Exercise 2.2:
- In order to establish a database table using the SQL CREATE command, follow these steps:
- From the database window for MATERIAL_MANAGER, click on Queries and then double click on Create query in Design View. The New Query window opens.
- The Show Table window will now be open. Close the window by clicking on Close.
- From the Toolbar, click on the SQL menu item. This will open a Query window that will allow you to enter a query in SQL.
- Delete any existing query in the query window and enter the following query:

```
CREATE TABLE PURCHASE_ORDER  
([PO_NUMBER]    SMALLINT,  
[RELEASE_DATE]  DATE,  
[PO_STATUS]     CHAR(6),  
[PO_AMT]        CURRENCY,  
[VENDOR_ID]     CHAR(5),  
CONSTRAINT [INDEX1] PRIMARY KEY (PO_NUMBER));
```
- Click the run button on the Toolbar to execute the query. The table is created.
- Close the query window (do not save the query) and click the Table tab on the database window.
- Confirm that a table with the name PURCHASE_ORDER now exists. The empty table can be opened by double clicking on the table name.

What is a Database Index?

- A database index is a data structure that improves the speed of data retrieval operations. Think of it as a roadmap that tells the database exactly where to find specific information without having to search through every record.
- Real-World Analogy: Phone Book
- Without Index (Sequential Search)
- To find “John Smith” in an unsorted list of 100,000 names:
 - Start at the first name
 - Check each name one by one
 - Continue until you find “John Smith”
 - Worst case: Check all 100,000 names
- With Index (Indexed Search)
- Names are alphabetically sorted with pointers:
 - Go directly to the “S” section
 - Find “Smith” entries
 - Locate “John Smith” immediately
 - Result: Found in just a few steps

Primary Keys and Indexes

- Automatic Index Creation
- When you create a primary key:
- `CONSTRAINT [INDEX2] PRIMARY KEY (VENDOR_ID)`
- What happens:
 1. Primary Key Constraint: Enforces uniqueness and non-null values
 2. Automatic Index: Database creates an index on `VENDOR_ID` for fast lookups
 3. Constraint Name: “INDEX2” is just a label for the constraint (not the index)

5. Structured Query Language

- Similarly, create the following tables using SQL commands:
- Table 1: VENDOR
- Primary Key: VENDOR_ID
- Table 2: PO_DETAIL
- Primary Key: (PO_NUMBER, PO_LINE_ITEM) - Composite Primary Key

SQL Commands

- CREATE TABLE VENDOR
([VENDOR_ID] CHAR(5),
[V_NAME] CHAR(20),
[V_STREET] CHAR(20),
[V_CITY] CHAR(20),
[V_STATE] CHAR(2),
[V_ZIP] CHAR(5),
CONSTRAINT [INDEX2] PRIMARY KEY (VENDOR_ID));

```
CREATE TABLE PO_DETAIL  
([PO_NUMBER] SMALLINT,  
[PO_LINE_ITEM] SMALLINT,  
[MATERIAL_ID] CHAR(10),  
[UNITS] CHAR(4),  
[QUANTITY] SINGLE,  
[BALANCE] SINGLE,  
[PROMISED_DEL_DATE] DATE,  
[UNIT_COST] CURRENCY,  
[STATUS] CHAR(6),  
CONSTRAINT [INDEX3]  
PRIMARY KEY (PO_NUMBER, PO_LINE_ITEM));
```

5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- • Once created, the tables must be populated in order to be useful
- • In this section we shall discuss and illustrate several keyword commands for populating and manipulating data in the database; in particular, the keywords INSERT, SELECT, UPDATE, and DELETE.

5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- INSERT Keyword
- The tables that were created are now empty. Records can be placed into these tables using the SQL INSERT command. The syntax is as follows:
- INSERT INTO <table name>
([<attribute1 name>], [<attribute2 name>],...)
VALUES (<value1>, <value2>,...);
- When all attribute columns of a table are being populated, it is unnecessary to define the attribute name. This is needed only when some specific attribute(s) of the table is being inserted

5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- INSERT Keyword
- Access Exercise 2.3:
- Return to the query window. Follow the steps of Access Exercise 2.2 if needed.
- Delete any existing query in the query window and enter the following query:
- ```
INSERT INTO PURCHASE_ORDER
VALUES (2591, "02/10/06", "CLOSED", 4300.00, "V110");
```
- Click the run button on the Toolbar and YES on the warning message to execute the query. The first record is now inserted into the table. You can view this record by closing the query window, returning to the database window, selecting the table name from the table objects window and clicking on Open.
- Following the same procedure, enter the remaining data for the PURCHASE\_ORDER table.

# Data to Insert

- Use the following reference tables to write INSERT statements for populating the VENDOR and PO\_DETAIL tables with the provided data.

# VENDOR Table Data

| VENDOR_ID | V_NAME               | V_STREET       | V_CITY       | V_STATE | V_ZIP |
|-----------|----------------------|----------------|--------------|---------|-------|
| V110      | Jersey Vegetable Co. | 2 Main St.     | Patterson    | NJ      | 07055 |
| V25       | General Provisions   | 125 Common St. | Boise        | ID      | 44530 |
| V250      | Spices Unlimited     | 25 Salty Lane  | East Hampton | NY      | 10027 |
| V75       | Pasta Supply, Inc.   | 34 Henry St.   | Philadelphia | PA      | 09098 |

# PURCHASE\_ORDER Table Data

| PO_NUMBER | RELEASE_DATE | PO_STATUS | PO_AMT     | VENDOR_ID |
|-----------|--------------|-----------|------------|-----------|
| 2591      | 2/10/06      | CLOSED    | \$4,300.00 | V110      |
| 2592      | 2/10/06      | OPEN      | \$505.50   | V25       |
| 2593      | 2/11/06      | OPEN      | \$4,000.00 | V110      |
| 2594      | 2/12/06      | OPEN      | \$3,260.00 | V250      |
| 2595      | 2/15/06      | OPEN      | \$500.00   | V250      |
| 2596      |              | HOLD      | \$1,000.00 | V75       |

# PO\_DETAIL Table Data

| PO_NUMBER | PO_LINE_ITEM | MATERIAL_ID | UNITS | QUANTITY | BALANCE | PROMISED_DEL_DATE | UNIT_COST | STATUS |
|-----------|--------------|-------------|-------|----------|---------|-------------------|-----------|--------|
| 2591      | 1            | RM201       | LB    | 1000.0   | 0       | 2/20/06           | \$2.00    | CLOSED |
| 2591      | 2            | RM202       | LB    | 1000.0   | 0       | 2/20/06           | \$2.00    | CLOSED |
| 2591      | 3            | RM205       | LB    | 300.0    | 0       | 2/20/06           | \$1.00    | CLOSED |
| 2592      | 1            | RM505       | GAL   | 600.5    | 0       | 2/25/06           | \$0.50    | CLOSED |
| 2592      | 2            | RM610       | GAL   | 210.5    | 210     | 3/10/06           | \$0.50    | OPEN   |
| 2594      | 1            | RM510       | LB    | 4000.0   | 4000    | 3/12/06           | \$0.50    | OPEN   |
| 2594      | 2            | RM511       | LB    | 2000.0   | 2000    | 3/12/06           | \$0.25    | OPEN   |
| 2594      | 3            | RM516       | LB    | 2000.0   | 2000    | 3/12/06           | \$0.25    | OPEN   |
| 2594      | 4            | RM540       | LB    | 560.0    | 560     | 3/20/06           | \$0.50    | OPEN   |



# SQL Commands - VENDOR

- ```
INSERT INTO VENDOR (VENDOR_ID, V_NAME, V_STREET, V_CITY, V_STATE, V_ZIP)
VALUES ('V110', 'Jersey Vegetable Co.', '2 Main St.', 'Patterson', 'NJ', '07055');
```



```
INSERT INTO VENDOR (VENDOR_ID, V_NAME, V_STREET, V_CITY, V_STATE, V_ZIP)
VALUES ('V25', 'General Provisions', '125 Common St.', 'Boise', 'ID', '44530');
```



```
INSERT INTO VENDOR (VENDOR_ID, V_NAME, V_STREET, V_CITY, V_STATE, V_ZIP)
VALUES ('V250', 'Spices Unlimited', '25 Salty Lane', 'East Hampton', 'NY', '10027');
```



```
INSERT INTO VENDOR (VENDOR_ID, V_NAME, V_STREET, V_CITY, V_STATE, V_ZIP)
VALUES ('V75', 'Pasta Supply, Inc.', '34 Henry St.', 'Philadelphia', 'PA', '09098');
```

SQL Commands - PURCHASE_ORDER

- ```
INSERT INTO PURCHASE_ORDER (PO_NUMBER, RELEASE_DATE, PO_STATUS, PO_AMT, VENDOR_ID)
VALUES (2591, '2/10/2006', 'CLOSED', 4300.00, 'V110');
```

  

```
INSERT INTO PURCHASE_ORDER (PO_NUMBER, RELEASE_DATE, PO_STATUS, PO_AMT, VENDOR_ID)
VALUES (2592, '2/10/2006', 'OPEN', 505.50, 'V25');
```

  

```
INSERT INTO PURCHASE_ORDER (PO_NUMBER, RELEASE_DATE, PO_STATUS, PO_AMT, VENDOR_ID)
VALUES (2593, '2/11/2006', 'OPEN', 4000.00, 'V110');
```

  

```
INSERT INTO PURCHASE_ORDER (PO_NUMBER, RELEASE_DATE, PO_STATUS, PO_AMT, VENDOR_ID)
VALUES (2594, '2/12/2006', 'OPEN', 3260.00, 'V250');
```

  

```
INSERT INTO PURCHASE_ORDER (PO_NUMBER, RELEASE_DATE, PO_STATUS, PO_AMT, VENDOR_ID)
VALUES (2595, '2/15/2006', 'OPEN', 500.00, 'V250');
```

  

```
INSERT INTO PURCHASE_ORDER (PO_NUMBER, RELEASE_DATE, PO_STATUS, PO_AMT, VENDOR_ID)
VALUES (2596, NULL, 'HOLD', 1000.00, 'V75');
```

# SQL Commands - PO\_DETAIL

- ```
INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2591, 1, 'RM201', 'LB', 1000.0, 0, '2/20/2006', 2.00, 'CLOSED');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2591, 2, 'RM202', 'LB', 1000.0, 0, '2/20/2006', 2.00, 'CLOSED');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2591, 3, 'RM205', 'LB', 300.0, 0, '2/20/2006', 1.00, 'CLOSED');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2592, 1, 'RM505', 'GAL', 600.5, 0, '2/25/2006', 0.50, 'CLOSED');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2592, 2, 'RM610', 'GAL', 210.5, 210, '3/10/2006', 0.50, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2594, 1, 'RM510', 'LB', 4000.0, 4000, '3/12/2006', 0.50, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2594, 2, 'RM511', 'LB', 2000.0, 2000, '3/12/2006', 0.25, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2594, 3, 'RM516', 'LB', 2000.0, 2000, '3/12/2006', 0.25, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2594, 4, 'RM540', 'LB', 560.0, 560, '3/20/2006', 0.50, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2593, 1, 'RM210', 'LB', 1000.0, 500, '2/25/2006', 2.00, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2593, 2, 'RM211', 'LB', 2000.0, 2000, '3/10/2006', 1.00, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2595, 1, 'RM305', 'LB', 400.0, 400, '2/27/2006', 0.50, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2595, 2, 'RM505', 'LB', 1200.0, 1200, '2/27/2006', 0.25, 'OPEN');

INSERT INTO PO_DETAIL (PO_NUMBER, PO_LINE_ITEM, MATERIAL_ID, UNITS, QUANTITY, BALANCE, PROMISED_DEL_DATE, UNIT_COST, STATUS)
VALUES (2596, 1, 'RM502', 'LB', 5000.0, 5000, NULL, 0.20, 'OPEN');
```

5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- SELECT Keyword
- The workhorse in retrieving and manipulating data in a table is the SELECT keyword, which can be used in many command variations. The syntax of a SELECT command is as follows:
- SELECT [DISTINCT] <attributes/*>
FROM <table name>
WHERE <condition>
ORDER BY <attribute name> ASC/DESC;

5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- SELECT Keyword
The functions of the various clauses shall be illustrated in the following examples. When the SELECT command is used, the minimum requirement is to identify the attributes to be selected and the table from which they are selected.
- The wildcard * identifies all attribute fields of the table. Thus, the command to retrieve the entire contents of the table PURCHASE_ORDER would be as follows:
- SELECT *
FROM PURCHASE_ORDER;
- On execution of this command, the entire contents of the table PURCHASE_ORDER is retrieved and displayed on the screen. Users can be more specific about the columns they wish to retrieve by specifying the column's attribute name in the SELECT clause

5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- SELECT Keyword
 - There may be more than one selection criterion for a retrieval
 - This can be handled by extending the WHERE clause using logical operators
 - The logical operators are AND, OR, and NOT
- For example, suppose we wish to look at all open orders that are above \$500. This would require the logical AND, since we want the orders that are open AND greater than \$500. The WHERE clause is extended using logical operators as follows:
- Click to show SQL solution
- ```
SELECT *
FROM PURCHASE_ORDER
WHERE PO_STATUS="OPEN"
AND PO_AMT>500;
```

## 5. Structured Query Language

MANAGING THE DATA IN THE DATABASE TABLE

Access Exercise 2.4:

Open the Query window and retrieve the following data from the table PURCHASE\_ORDER

PO\_NUMBER, PO\_STATUS, PO\_AMT

After execution, the table of retrieved data should appear as shown in Figure 2.4.

| PO_NUMBER | PO_STATUS | PO_AMT     |
|-----------|-----------|------------|
| 2591      | CLOSED    | \$4,300.00 |
| 2592      | OPEN      | \$505.50   |
| 2593      | OPEN      | \$4,000.00 |
| 2594      | OPEN      | \$3,260.00 |
| 2595      | OPEN      | \$500.00   |
| 2596      | HOLD      | \$1,000.00 |

Result of Access Exercise 2.4

- Click to show SQL solution
- ```
SELECT PO_NUMBER, PO_STATUS, PO_AMT  
FROM PURCHASE_ORDER;
```


5. Structured Query Language

MANAGING THE DATA IN THE DATABASE TABLE

Access Exercise 2.5:

Open the Query window and retrieve the following data from the table PURCHASE_ORDER, but order by PO_AMT (smallest to largest)

PO_NUMBER, PO_STATUS, PO_AMT

After execution, the table of retrieved data should appear as shown in Figure 2.5.

PO_NUMBER	PO_STATUS	PO_AMT
2595	OPEN	\$500.00
2592	OPEN	\$505.50
2596	HOLD	\$1,000.00
2594	OPEN	\$3,260.00
2593	OPEN	\$4,000.00
2591	CLOSED	\$4,300.00

Result of Access Exercise 2.5

- note

The default is to use ascending order. For descending order, the keyword DESC is used following the attribute name.

- Click to show SQL solution
- ```
SELECT PO_NUMBER, PO_STATUS, PO_AMT
FROM PURCHASE_ORDER
ORDER BY PO_AMT;
```

## 5. Structured Query Language

MANAGING THE DATA IN THE DATABASE TABLE

SELECT Keyword

Access Exercise 2.6:

Open the Query window and retrieve the following data from the table PURCHASE\_ORDER, order by VENDOR\_ID, and within each vendor group, order by PO\_AMT in descending order

VENDOR\_ID, PO\_AMT, PO\_NUMBER, PO\_STATUS

After execution, the table of retrieved data should appear as shown in Figure 2.6.

| VENDOR_ID | PO_AMT     | PO_NUMBER | PO_STATUS |
|-----------|------------|-----------|-----------|
| V110      | \$4,300.00 | 2591      | CLOSED    |
| V110      | \$4,000.00 | 2593      | OPEN      |
| V25       | \$505.50   | 2592      | OPEN      |
| V250      | \$3,260.00 | 2594      | OPEN      |
| V250      | \$500.00   | 2595      | OPEN      |
| V75       | \$1,000.00 | 2596      | HOLD      |

Results of Access Exercise 2.6

- Click to show SQL solution
- ```
SELECT VENDOR_ID, PO_AMT, PO_NUMBER, PO_STATUS  
FROM PURCHASE_ORDER  
ORDER BY VENDOR_ID, PO_AMT DESC;
```

5. Structured Query Language

MANAGING THE DATA IN THE DATABASE TABLE

SELECT Keyword

The DISTINCT keyword in the SELECT clause allows the user to sort a single column of a table and to return a list of the unique entries in that column. In effect, the resulting list does not contain redundancies.

For example, the following command will return the unique VENDOR_IDs in the PURCHASE_ORDERS table:

```
SELECT DISTINCT VENDOR_ID  
FROM PURCHASE_ORDER;
```

VENDOR_ID
V110
V25
V250
V75

5. Structured Query Language

MANAGING THE DATA IN THE DATABASE TABLE

SELECT Keyword

Access Exercise 2.7:

Open the Query window and retrieve the following data from the table PURCHASE_ORDER where the PO_AMT is more than 1000. Order by PO_AMT in ascending order

VENDOR_ID, PO_NUMBER, PO_AMT

After execution, the table of retrieved data should appear as shown in Figure 2.7.

VENDOR_ID	PO_NUMBER	PO_AMT
V250	2594	\$3,260.00
V110	2593	\$4,000.00
V110	2591	\$4,300.00

Results of Access Exercise 2.7

- Click to show SQL solution
- ```
SELECT VENDOR_ID, PO_NUMBER, PO_AMT
FROM PURCHASE_ORDER
WHERE PO_AMT > 1000
ORDER BY PO_AMT;
```

## 5. Structured Query Language

MANAGING THE DATA IN THE DATABASE TABLE

SELECT Keyword

Access Exercise 2.8:

Open the Query window and retrieve the following data from the table PURCHASE\_ORDER

```
SELECT *
FROM PURCHASE_ORDER
WHERE PO_STATUS="OPEN"
AND (VENDOR_ID>"V150" AND PO_AMT>500)
ORDER BY PO_AMT;
```

After execution, the retrieved data should appear as shown in Figure 2.8.

| PO_NUMB<br>ER | RELEASE_D<br>ATE | PO_STATUS | PO_AMT    | VENDOR_I<br>D |
|---------------|------------------|-----------|-----------|---------------|
| 2592          | 2/10/06          | OPEN      | \$505.50  | V25           |
| 2594          | 2/12/06          | OPEN      | \$3260.00 | V250          |

Results of Access Exercise 2.8



- note  
There are more keywords that you can use. LIKE, BETWEEN, NULL.
- Create queries using these keywords to understand each one.

# 5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- UPDATE Keyword
- The UPDATE keyword allows the user to replace existing values in a table with new values. For example, if a vendor were to change the promised delivery date of an order, it would be desirable to reflect this change immediately in the appropriate table of the database
- The syntax of the UPDATE command is as follows:
- UPDATE <table name>  
SET <attribute name> = <value/expression> [...]  
[WHERE <condition>];
- The updated field value may be a new value or any admissible expression, typically an arithmetic expression. So, for example, if the attribute is the price of a product and the company raises all its prices by 2%, the SET clause could read as follows:
- SET PRICE = PRICE \* 1.02

# 5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- UPDATE Keyword
- Access Exercise 2.9:
- Vendor V250 has just informed us that all items on purchase order number 2594 will not be delivered until 03/22/06.
- Open the Query window and perform the data update in the table PO\_DETAIL.
- UPDATE PO\_DETAIL  
SET PROMISED\_DEL\_DATE = "03/22/06"  
WHERE PO\_NUMBER=2594;
- After execution, open the table PO\_DETAIL and confirm that the changes have occurred.

# 5. Structured Query Language

- MANAGING THE DATA IN THE DATABASE TABLE
- DELETE Keyword
- The DELETE keyword allows the user to remove one or more rows from a table (i.e., delete one or more records). The syntax for the DELETE command is as follows:
- DELETE FROM <table name>  
WHERE <condition>;
- So, for example, if it was determined that purchase order number 2596 would not be released because the requisitioning department reconsidered the purchase, we may want to delete that record from the database. This can be accomplished with the following commands:
- DELETE FROM PO\_DETAIL  
WHERE PO\_NUMBER = 2596;  
  
DELETE FROM PURCHASE\_ORDER  
WHERE PO\_NUMBER = 2596;

# 5. Structured Query Language

- CONVERTING DATA INTO INFORMATION
- The purpose of having data is to be able to summarize them in ways that support decision making
- Aggregate Functions in SQL
- The aggregate functions allow the user to specify a summary mathematical operation with a keyword.
- The basic aggregate functions of the SQL are AVG, SUM, MIN, MAX, and COUNT.
- The syntax of the aggregate functions is as follows:
- SELECT AGGREGATE FUNCTION ([DISTINCT] <attribute name>)  
FROM <table name>  
WHERE <condition>  
GROUP BY <attribute name> [HAVING <condition>];

## 5. Structured Query Language

- CONVERTING DATA INTO INFORMATION
- Suppose we are interested in the average dollar value of open purchase orders.
- ```
SELECT AVG(PO_AMT)
FROM PURCHASE_ORDER
WHERE PO_STATUS="OPEN";
```
- Suppose we want the minimum of PO_AMT
- ```
SELECT MIN(PO_AMT) AS MIN_AMT
FROM PURCHASE_ORDER;
```

# 5. Structured Query Language

- Grouping Data
- The GROUP BY clause can be used to group data from an aggregate function by a column attribute. This allows you to display the results of several aggregates by some meaningful summary.
- For example, suppose you want to summarize aggregates of data from the PO\_DETAIL table by PO\_NUMBER.
- Consider the following example:
  - ```
SELECT PO_NUMBER, MIN(QUANTITY), MAX(QUANTITY)
FROM PO_DETAIL
GROUP BY PO_NUMBER;
```
- - A subclause of the GROUP BY clause is the HAVING clause.
 - This clause allows the programmer to place a condition or filter on the group.
- For example, suppose we are interested in the maximum and minimum quantities of those purchase orders that have a maximum quantity greater than 1000.
- This can be accomplished by applying the HAVING clause as follows:
 - ```
SELECT PO_NUMBER, MIN(QUANTITY), MAX(QUANTITY)
FROM PO_DETAIL
GROUP BY PO_NUMBER HAVING MAX(QUANTITY)>1000;
```

# 5. Structured Query Language

- Subqueries in SQL
- Subqueries allow the user to condition one query on the results of another query from a table and also allow the user to retrieve information in a table based on the results of a query in another table.
- As an example of a subquery on a single table, suppose that you want to list all purchase order numbers that have amounts that are greater than the average amount of all purchase orders.
- Query 1:
- ```
SELECT AVG(PO_AMT)
FROM PURCHASE_ORDER;
```
- Query 2:
- ```
SELECT PO_NUMBER, PO_AMT
FROM PURCHASE_ORDER
WHERE PO_AMT > (result of query 1);
```
- The query would be:
- ```
SELECT PO_NUMBER, PO_AMT
FROM PURCHASE_ORDER
WHERE PO_AMT > (SELECT AVG(PO_AMT)
FROM PURCHASE_ORDER);
```


5. Structured Query Language

- Subqueries in SQL
- A subquery can also be used to retrieve related data in more than one table. For example, suppose you are interested in seeing the details of all the purchase orders that have been placed with vendor V250. These details are listed in the table PO_DETAIL. However, the table PO_DETAIL does not contain any reference to VENDOR_ID V250.
- On the other hand, V250 appears in the table PURCHASE_ORDER, and that table is related to PO_DETAIL by the attribute they have in common: PO_NUMBER.
- ```
SELECT *
FROM PO_DETAIL
WHERE PO_NUMBER IN (SELECT PO_NUMBER
FROM PURCHASE_ORDER
WHERE VENDOR_ID = "v250");
```

# 5. Structured Query Language

- Appending Tables Using Joins
- Note that the subquery in the previous section used the IN operator to return information from one table as criterion for retrieving from another table. However, the displayed information came only from the table referenced in the main query.  
There are times when it is desirable to display information from more than one table on the same retrieval.
- For example, one might want to display the `VENDOR_ID` and `RELEASE_DATE` from the `PURCHASE_ORDER` table along with the related details from the `PO_DETAIL` table. This is the purpose of a table join. The syntax for a join is as follows:
- `SELECT <table1 name.attribute name>, <table2 name.attribute name>,...`  
`FROM <table1 name>, <table2 name>,...`  
`WHERE <join condition>`  
`ORDER BY <column name>`

# 5. Structured Query Language

Access Exercise 2.13:

Open the Query window and perform the following query which joins data from PURCHASE\_ORDER and PO\_DETAIL tables.

```
SELECT PURCHASE_ORDER.VENDOR_ID,
 PURCHASE_ORDER.RELEASE_DATE,
 PURCHASE_ORDER.PO_NUMBER,
 PO_DETAIL.*
FROM PURCHASE_ORDER, PO_DETAIL
WHERE PURCHASE_ORDER.PO_NUMBER =
 PO_DETAIL.PO_NUMBER
AND PURCHASE_ORDER.VENDOR_ID="V250";
```

The result should appear as in Figure 2.12, below.

| VE<br>ND<br>OR<br>_ID | RELE<br>ASE_<br>DAT<br>E | PO<br>_N<br>U<br>MB<br>ER | P<br>O<br>_N<br>U<br>M | P<br>O<br>_L<br>IN<br>E | MAT<br>ERI<br>AL_I<br>D | U<br>N<br>I<br>T<br>S | Q<br>U<br>A<br>NT<br>ITY | B<br>A<br>L<br>A<br>N<br>C<br>E | PROMI<br>SED_DE<br>L_DATE | UN<br>IT_<br>CO<br>ST | S<br>T<br>A<br>T<br>U<br>S |
|-----------------------|--------------------------|---------------------------|------------------------|-------------------------|-------------------------|-----------------------|--------------------------|---------------------------------|---------------------------|-----------------------|----------------------------|
| V2<br>50              | 2/12<br>/06              | 25<br>94                  | 2<br>5<br>9<br>4       | 1                       | RM<br>510               | L<br>B                | 40<br>00                 | 4<br>0<br>0<br>0                | 3/12/0<br>6               | \$0.<br>50            | O<br>P<br>E<br>N           |
| V2<br>50              | 2/12<br>/06              | 25<br>94                  | 2<br>5<br>9<br>4       | 2                       | RM<br>511               | L<br>B                | 20<br>00                 | 2<br>0<br>0<br>0                | 3/12/0<br>6               | \$0.<br>25            | O<br>P<br>E<br>N           |
| V2<br>50              | 2/12<br>/06              | 25<br>94                  | 2<br>5<br>9<br>4       | 3                       | RM<br>516               | L<br>B                | 20<br>00                 | 2<br>0<br>0<br>0                | 3/12/0<br>6               | \$0.<br>25            | O<br>P<br>E<br>N           |
| V2<br>50              | 2/12<br>/06              | 25<br>94                  | 2<br>5<br>9<br>4       | 4                       | RM<br>540               | L<br>B                | 56<br>0                  | 5<br>6<br>0                     | 3/20/0<br>6               | \$0.<br>50            | O<br>P<br>E                |

Result of Access Exercise 2.13

# To Conclude

- • The relational database model can be viewed by a user as a set of tables. The tables are defined by an entity set name and contains the values of attributes for instances of that entity set. Each instance in a table is uniquely identified by a primary key. Tables are related to each other through the use of foreign keys.
- • The programming language of relational databases is Structured Query Language.
- • In this chapter, we demonstrated the use of SQL to give the reader some concept of how tables are related to each other in the relational design.
- • An information system design project begins by establishing the requirements of the design project. The next chapters focus on this aspect of the design problem by demonstrating the use of design methodologies at the conceptual level.

# HW

- On Blackboard Due next week!